

ECOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

École Normale Supérieure de
l'Enseignement Technique

Department of Information and Mathematics

IT Engineering, Cybersecurity and Digital Trust engineering

Module : Sécurité des systèmes distribués

Devoir d'application : Authentification et autorisation avec Spring Security, JWT et React

---

Élèves :
Yasser NAMEZ

Enseignant :
Pr. Abdelmajid
BOUSSELHAM

28 octobre 2026

Table des matières

1 Introduction **2**
1.1 Technologies utilisées . . . 2
1.2 Lancement de l'application . . . 2

2 Architecture générale **2**

3 Modèle de données **4**
3.1 Diagramme des entités . . . 4
3.2 Travail de recherche : relations JPA . . . 4

4 Authentification et JWT **5**
4.1 Endpoints publics . . . 5
4.2 Travail de recherche : les claims JWT . . . 5

5 Configuration de Spring Security **6**
5.1 Le filtre JWT . . . 6

6 Gestion des rôles **7**
6.1 Travail de recherche : sécurisation par rôle . . . 7
6.2 Endpoints de l'API . . . 8

7 Contrôle du propriétaire d'un incident **8**
7.1 Règles métier . . . 9
7.2 Historique et notes de traitement . . . 9

8 Validation des données **9**

9 Gestion des exceptions **10**
9.1 Travail de recherche : codes HTTP . . . 10

10 Frontend React **10**
10.1 Structure . . . 10
10.2 Gestion du JWT . . . 11
10.3 Expiration du JWT . . . 11
10.4 Protection des routes . . . 11
10.5 Dashboards . . . 11

11 Access Token et Refresh Token **12**

12 Tests **12**
12.1 Scénario fonctionnel . . . 12
12.2 Tests de sécurité avec Postman . . . 15

1 Introduction

Ce devoir consiste à développer une application web de déclaration et de suivi d'incidents informatiques et de cybersécurité (malware, phishing, panne réseau, fuite de données, etc.). L'application gère trois types d'utilisateurs : `EMPLOYEE`, `TECHNICIAN` et `ADMIN`.

Objectifs

- Mettre en place l'authentification avec JWT.
- Gérer les rôles et protéger les routes du backend et de React.
- Contrôler l'accès aux données (propriétaire d'un incident).
- Valider les données et gérer proprement les erreurs.
- Fournir un tableau de bord et un historique des incidents.

1.1 Technologies utilisées

- **Backend** : Spring Boot 3, Spring Security, Spring Data JPA, Bean Validation, jjwt, base H2.
- **Frontend** : React (Vite), React Router, Axios.
- **Tests** : Postman.

1.2 Lancement de l'application

```
# Backend (port 8080)
cd backend
mvn spring-boot:run

# Frontend (port 5173)
cd frontend
npm install
npm run dev
```

Trois comptes de test sont créés au démarrage du backend :

Rôle	Email	Mot de passe
ADMIN	`admin@test.com`	`admin123`
TECHNICIAN	`tech@test.com`	`tech123`
EMPLOYEE	`employee@test.com`	`emp123`

2 Architecture générale

L'application est composée de deux projets : un frontend React et un backend Spring Boot. Le frontend envoie des requêtes HTTP/JSON avec le header `Authorization: Bearer JWT`. Dans le backend, chaque requête traverse le filtre JWT de Spring Security, puis les controllers, les services et les repositories avant d'atteindre la base de données.

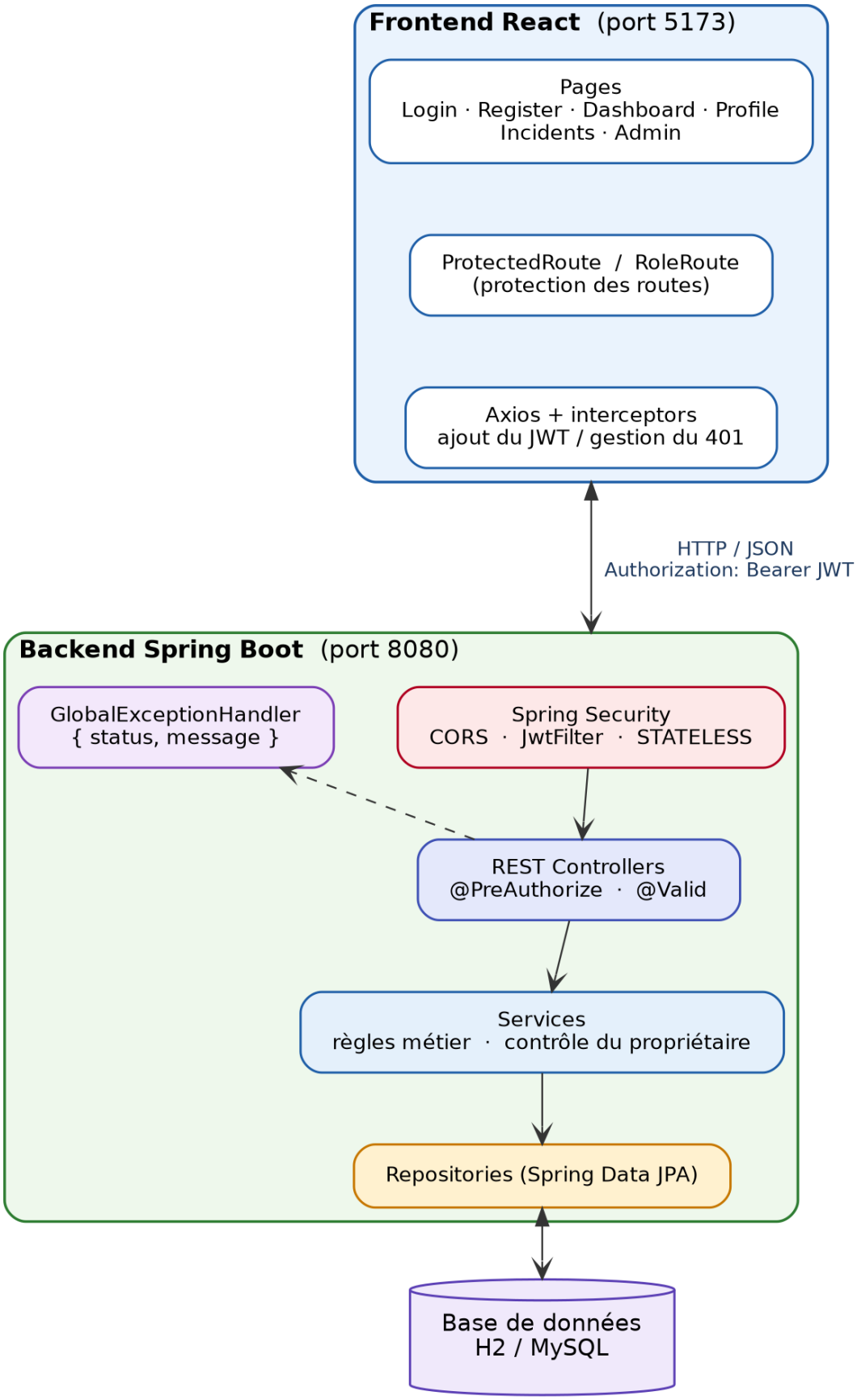


FIGURE 1 – Architecture générale de l'application

3 Modèle de données

3.1 Diagramme des entités

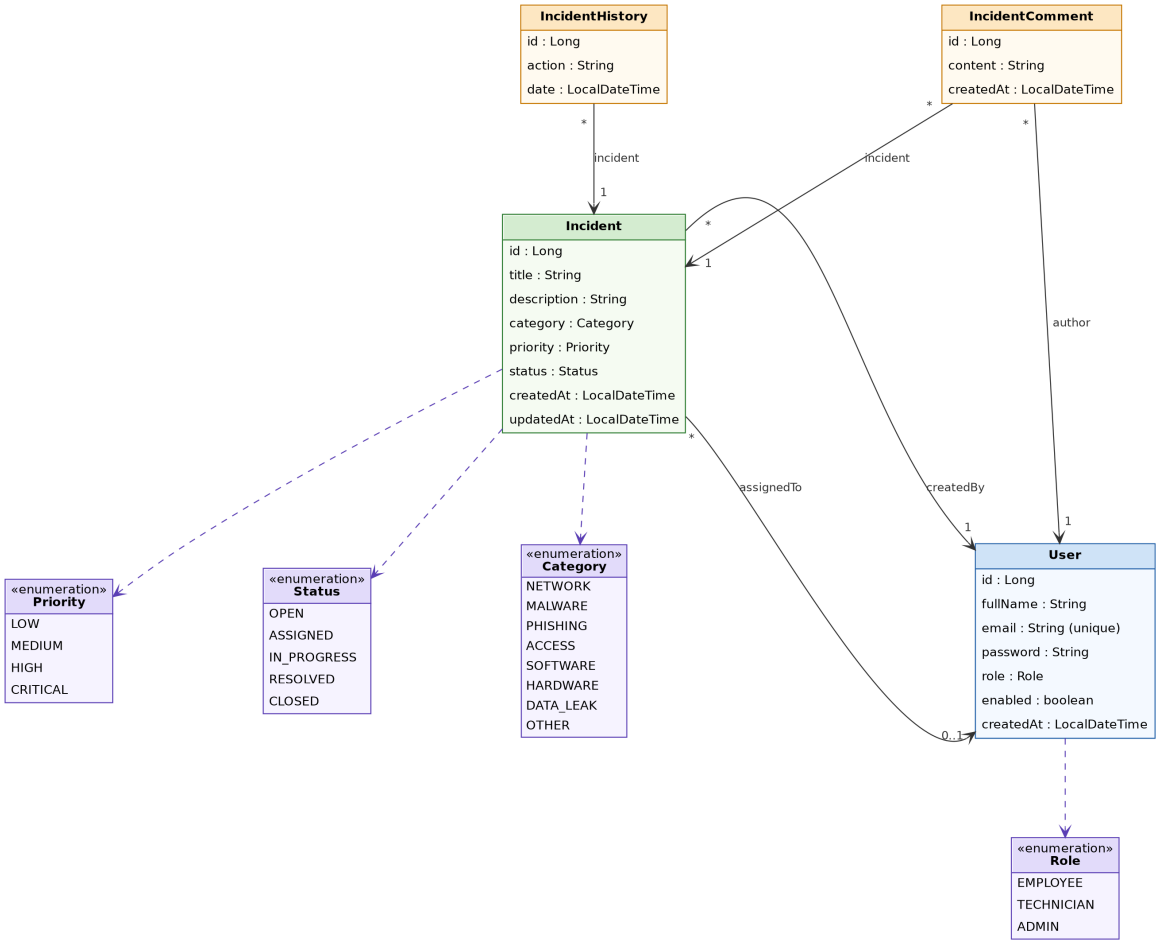


FIGURE 2 – Diagramme des entités

3.2 Travail de recherche : relations JPA

`@ManyToOne` et `@OneToMany`

`@ManyToOne` indique que plusieurs objets d'une classe sont liés à un seul objet d'une autre classe. Dans la base, la clé étrangère est placée dans la table qui porte l'annotation. `@OneToMany` est le côté inverse : un objet possède une liste d'objets, avec `mappedBy` pour indiquer le champ `@ManyToOne` correspondant.

Dans notre projet, plusieurs incidents sont liés à un utilisateur :

```
@ManyToOne(fetch = FetchType.LAZY)
private User createdBy;

@ManyToOne(fetch = FetchType.LAZY)
private User assignedTo;
```

LAZY ou EAGER ?

	Fonctionnement	Inconvénient
EAGER	L'objet lié est chargé immédiatement, avec l'entité principale.	Charge des données inutiles et ajoute des jointures.
LAZY	L'objet lié n'est chargé que lors du premier accès.	Provoque une `LazyInitializationException` hors transaction.

Justification de nos choix :

— Nous utilisons `LAZY` partout : l'utilisateur lié à un incident n'est chargé que si on en a besoin. Par défaut, `@ManyToOne` est EAGER, d'où le choix explicite de LAZY.

— Le problème du LAZY est résolu en convertissant les entités en DTO à l'intérieur d'un service annoté `@Transactional`. Les entités ne sont jamais renvoyées directement en JSON.

— Nous n'avons pas utilisé de `@OneToMany` (pas de liste d'incidents dans `User`) : ce n'est pas nécessaire, et cela évite les boucles de sérialisation JSON. L'historique et les notes sont récupérés avec des requêtes de repository.

4 Authentification et JWT

4.1 Endpoints publics

Méthode	Endpoint	Description
POST	`/auth/register`	Créer un compte (rôle EMPLOYEE)
POST	`/auth/login`	Se connecter, retourne `token`, `role`, `fullName`

Lors de l'inscription, le rôle est imposé par le backend (`Role.EMPLOYEE`). L'objet `RegisterRequest` ne contient pas de champ `role` : l'utilisateur ne peut donc pas choisir son rôle. Les mots de passe sont encodés avec **BCrypt**.

4.2 Travail de recherche : les claims JWT

Qu'est-ce qu'un claim ?

Un JWT est composé de trois parties : *header*, *payload* et *signature*. Le payload contient des **claims**, c'est-à-dire des informations sur l'utilisateur et sur le token. Il existe des claims standards (`sub` : sujet, `iat` : date de création, `exp` : date d'expiration) et des claims personnalisés, comme le rôle.

Notre token contient : l'email (`sub`), le rôle (claim `role`), la date de création (`iat`) et la date d'expiration (`exp`).

Ajouter un rôle dans le token :

```
Jwts.builder()
    .subject(email)
    .claim("role", role)
    .issuedAt(now)
    .expiration(new Date(now.getTime() + expirationMs))
    .signWith(key)
    .compact();
```

Récupérer un claim :

```
Claims claims = Jwts.parser().verifyWith(key).build()
        .parseSignedClaims(token).getPayload();
String email = claims.getSubject();
String role = claims.get("role", String.class);
```

Vérifier l'expiration : la méthode `parseSignedClaims` vérifie automatiquement la signature et la date d'expiration. Si le token est expiré, elle lève une `ExpiredJwtException` (sous-classe de `JwtException`). Si le token est invalide, une autre `JwtException` est levée. Dans les deux cas, le filtre n'authentifie pas la requête et l'API répond **401**.

La durée de vie du token est de **5 minutes** (propriété `jwt.expiration-minutes`), ce qui permet de tester facilement l'expiration.

5 Configuration de Spring Security

- Routes publiques : `/auth/**` et `/h2-console/**`.
- Routes protégées : `/api/**` (token obligatoire).
- `/api/admin/**` : réservé au rôle ADMIN.
- Sessions : `SessionCreationPolicy.STATELESS` (aucune session côté serveur).
- CORS : autorise l'origine `http://localhost:5173`.

```
http
  .csrf(csrf -> csrf.disable())
  .cors(Customizer.withDefaults())
  .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.
      STATELESS))
  .authorizeHttpRequests(auth -> auth
      .requestMatchers("/auth/**").permitAll()
      .requestMatchers(new AntPathRequestMatcher("/h2-console/**")).permitAll()
      .requestMatchers("/api/admin/**").hasRole("ADMIN")
      .requestMatchers("/api/**").authenticated()
      .anyRequest().authenticated())
  .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
```

5.1 Le filtre JWT

Le filtre `JwtFilter` s'exécute à chaque requête :

1. il lit le header `Authorization: Bearer ...` ;
2. il vérifie le token et extrait l'email et le rôle ;

3. il vérifie que le compte existe et qu'il est activé (`enabled`) ;
4. il place l'utilisateur et son rôle dans le `SecurityContextHolder`.

Si le token est absent, invalide ou expiré, aucune authentification n'est créée et Spring Security renvoie une réponse **401** en JSON.

6 Gestion des rôles

Un JWT valide ne donne pas accès à toutes les fonctionnalités : chaque action est contrôlée selon le rôle.

Fonctionnalité	EMPLOYEE	TECHNICIAN	ADMIN
Consulter / modifier son profil	✓	✓	✓
Créer un incident	✓	✗	✗
Consulter ses propres incidents	✓	✗	✗
Consulter les incidents affectés	✗	✓	✗
Modifier le statut d'un incident	✗	✓ (les siens)	✓
Ajouter une note de traitement	✗	✓ (les siens)	✗
Consulter tous les incidents	✗	✗	✓
Affecter un technicien	✗	✗	✓
Gérer les utilisateurs	✗	✗	✓
Consulter les statistiques	✗	✗	✓

6.1 Travail de recherche : sécurisation par rôle

— `hasRole('ADMIN')` vérifie que l'utilisateur possède l'autorité `ROLE_ADMIN` (le préfixe `ROLE_` est ajouté automatiquement).

— `hasAuthority('ROLE_ADMIN')` compare la chaîne exacte, sans ajouter de préfixe.

— `@PreAuthorize` applique une règle d'accès sur une méthode, avant son exécution.

— `@EnableMethodSecurity` active la prise en compte de `@PreAuthorize`.

Dans le projet, le rôle est déclaré dans le token, puis transformé en autorité `ROLE_xxx` par le filtre. Les controllers utilisent ensuite `@PreAuthorize` :

```
@PostMapping
@PreAuthorize("hasRole('EMPLOYEE')")
public IncidentResponse create(@Valid @RequestBody CreateIncidentRequest
    request) { ... }

@PutMapping("/{id}/assign/{technicianId}")
@PreAuthorize("hasRole('ADMIN')")
public IncidentResponse assign(@PathVariable Long id, @PathVariable Long
    technicianId) { ... }
```

6.2 Endpoints de l'API

Méthode	URL	Accès
GET / PUT	`/api/profile`	Authentifié
PUT	`/api/profile/password`	Authentifié
POST	`/api/incidents`	EMPLOYEE
GET	`/api/incidents/my`	EMPLOYEE
GET	`/api/incidents/assigned`	TECHNICIAN
GET	`/api/incidents`	ADMIN
GET	`/api/incidents/{id}`	Propriétaire ou rôle autorisé
PUT	`/api/incidents/{id}/status`	TECHNICIAN ou ADMIN
PUT	`/api/incidents/{id}/assign/{technicianId}`	ADMIN
POST	`/api/incidents/{id}/comments`	TECHNICIAN
GET	`/api/admin/users`	ADMIN
PUT	`/api/admin/users/{id}/enable`	ADMIN
PUT	`/api/admin/users/{id}/disable`	ADMIN
GET	`/api/admin/statistics`	ADMIN

7 Contrôle du propriétaire d'un incident

Un employé peut tester directement l'URL `GET /api/incidents/12` dans Postman, même si l'incident ne lui appartient pas. Masquer un bouton dans React ne suffit donc pas : le backend doit vérifier l'accès.

Solution : la classe `CurrentUser` récupère l'utilisateur connecté grâce à `SecurityContextHolder` :

```
String email = SecurityContextHolder.getContext()
       .getAuthentication().getName();
User user = userRepository.findByEmail(email).orElseThrow(...);
```

Ensuite, la méthode `checkAccess` du service compare cet utilisateur au propriétaire de l'incident :

```
private void checkAccess(Incident incident, User user) {
    boolean allowed = switch (user.getRole()) {
        case ADMIN -> true;
        case EMPLOYEE -> incident.getCreatedBy().getId().equals(user.getId());
        case TECHNICIAN -> incident.getAssignedTo() != null
               && incident.getAssignedTo().getId().equals(user.getId());
    };
    if (!allowed) {
        throw new ApiException(HttpStatus.FORBIDDEN, "Acces interdit");
    }
}
```

Cette méthode est appelée pour consulter un incident, changer son statut et ajouter une note. Un employé qui demande l'incident d'un autre reçoit donc une réponse **403**.

7.1 Règles métier

Règle	Implémentation
Nouvel utilisateur = EMPLOYEE	`AuthService.register` impose le rôle
Impossible de choisir ADMIN	Pas de champ `role` dans l'inscription
Mots de passe en BCrypt	Bean `BCryptPasswordEncoder`
Employé : ses incidents uniquement	Requête par `createdBy` et `checkAccess`
Technicien : incidents affectés uniquement	`checkAccess` (cas TECHNICIAN)
Seul l'admin affecte un technicien	`@PreAuthorize("hasRole('ADMIN')")`
Compte désactivé : connexion refusée	Test dans `AuthService.login` et dans `JwtFilter`
Incident CLOSED non modifiable	Test dans `updateStatus` (sauf ADMIN)

7.2 Historique et notes de traitement

Chaque changement important est enregistré dans l'entité `IncidentHistory` (création, affectation, changement de statut). Les techniciens ajoutent des notes de traitement, enregistrées dans `IncidentComment`. Les deux sont affichées dans la page de détail d'un incident.

8 Validation des données

Les données reçues sont validées par le backend avec Bean Validation. Le controller utilise `@Valid` sur le corps de la requête, et les annotations sont placées dans les DTO.

Annotation	Rôle
`@NotBlank`	Le texte ne doit pas être vide (titre, description, mot de passe)
`@NotNull`	La valeur est obligatoire (priorité, catégorie, statut)
`@Email`	Le format de l'email doit être valide
`@Size(min = 6)`	Longueur minimale du mot de passe
`@Valid`	Déclenche la validation sur l'objet reçu

```
public record CreateIncidentRequest(
    @NotBlank(message = "Le titre est obligatoire") String title,
    @NotBlank(message = "La description est obligatoire") String description,
    @NotNull(message = "La categorie est obligatoire") Category category,
    @NotNull(message = "La priorite est obligatoire") Priority priority) {}
```

☞ Attention

La validation faite dans React n'est qu'une aide pour l'utilisateur. La vraie validation est toujours faite par le backend.

9 Gestion des exceptions

Les erreurs sont centralisées dans une classe `GlobalExceptionHandler` annotée `@RestControllerAdv` (version de `@ControllerAdvice` qui renvoie directement du JSON). Chaque méthode annotée `@ExceptionHandler` gère un type d'erreur.

Une seule exception métier, `ApiException`, contient le statut HTTP et le message. Toutes les erreurs ont le même format :

```
{ "status": 404, "message": "Incident introuvable" }
```

Cas géré	Code	Message
Utilisateur introuvable	404	Utilisateur introuvable
Incident introuvable	404	Incident introuvable
Email déjà utilisé	409	Email déjà utilisé
Mauvais mot de passe (connexion)	401	Email ou mot de passe incorrect
Compte désactivé	403	Compte désactivé
Accès interdit	403	Accès interdit
Données invalides	400	Message de la validation

9.1 Travail de recherche : codes HTTP

Code	Signification	Exemple dans le projet
400 Bad Request	La requête est invalide.	Titre vide, JSON mal formé
401 Unauthorized	L'utilisateur n'est pas authentifié.	Pas de token, token invalide ou expiré
403 Forbidden	L'utilisateur est authentifié mais n'a pas le droit.	Employé sur une route ADMIN
404 Not Found	La ressource n'existe pas.	Incident introuvable

La différence importante est entre **401** (« qui êtes-vous ? ») et **403** (« je sais qui vous êtes, mais vous n'avez pas le droit »).

10 Frontend React

10.1 Structure

```
src/
  components/ Navbar, ProtectedRoute, RoleRoute, IncidentTable
  pages/      Login, Register, Dashboard, Profile, MyIncidents, NewIncident,
              IncidentDetails, AssignedIncidents, AdminUsers, AdminIncidents
  services/   api.js, authService.js, incidentService.js
  App.jsx, main.jsx
```

10.2 Gestion du JWT

Après la connexion, le token, le rôle et le nom sont stockés dans le `localStorage`. Un **interceptor Axios** ajoute automatiquement le header `Authorization` à chaque requête :

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = "Bearer " + token;
  }
  return config;
});
```

10.3 Expiration du JWT

Un second interceptor traite les réponses. Quand le backend renvoie **401**, le token est supprimé et l'utilisateur est redirigé vers la page Login avec le message « Votre session a expiré ». Les erreurs 401 de `/auth/*` (mauvais mot de passe) sont ignorées pour ne pas afficher ce message à tort.

```
api.interceptors.response.use(
  (response) => response,
  (error) => {
    const isAuthRequest = error.config?.url?.startsWith("/auth/");
    if (error.response?.status === 401 && !isAuthRequest) {
      localStorage.clear();
      window.location.href = "/login?expired=1";
    }
    return Promise.reject(error);
  }
);
```

10.4 Protection des routes

- `ProtectedRoute` : si l'utilisateur n'a pas de token, il est redirigé vers `/login`.
- `RoleRoute` : si le rôle n'est pas dans la liste autorisée, l'utilisateur est redirigé vers l'accueil. Exemple : `/admin/users` n'est accessible qu'à un ADMIN.

☞ Attention

La protection des routes React améliore l'expérience utilisateur, mais la véritable sécurité est assurée par le backend.

10.5 Dashboards

- **EMPLOYEE** : nombre total d'incidents, OPEN, IN_PROGRESS, RESOLVED/-CLOSED et les derniers incidents.
- **TECHNICIAN** : incidents affectés, critiques, en cours et résolus.
- **ADMIN** : utilisateurs, employés, techniciens, incidents, OPEN, IN_PROGRESS et CLOSED (endpoint `/api/admin/statistics`).

11 Access Token et Refresh Token

Le Refresh Token est une partie optionnelle du devoir et n'a pas été implémenté. Voici le résultat de la recherche.

	Access Token	**Refresh Token**
Rôle	Envoyé à chaque requête pour accéder à l'API.	Sert uniquement à obtenir un nouvel Access Token.
Durée de vie	Courte (quelques minutes).	Longue (jours ou semaines).
Risque	Limité, car il expire vite.	Plus élevé, il doit être mieux protégé.

Fonctionnement : à la connexion, le serveur renvoie les deux tokens. Quand l'Access Token expire, le client envoie le Refresh Token à `POST /auth/refresh` et reçoit un nouvel Access Token, sans que l'utilisateur ressaisisse son mot de passe.

Déconnexion : le client supprime les deux tokens, et le serveur doit invalider (supprimer ou marquer comme révoqué) le Refresh Token enregistré, pour qu'il ne puisse plus servir.

12 Tests

12.1 Scénario fonctionnel

1. Création de deux comptes EMPLOYEE, d'un TECHNICIAN et d'un ADMIN.
2. Le premier employé déclare deux incidents, le deuxième employé en déclare un.
3. Le premier employé ne peut pas consulter l'incident du deuxième.
4. L'admin consulte tous les incidents et affecte un incident au technicien.
5. Le technicien consulte ses incidents, passe le statut à `IN_PROGRESS` puis `RESOLVED`, et ajoute une note.
6. L'employé se reconnecte et vérifie le nouvel état et l'historique de son incident.



FIGURE 3 – Page de connexion

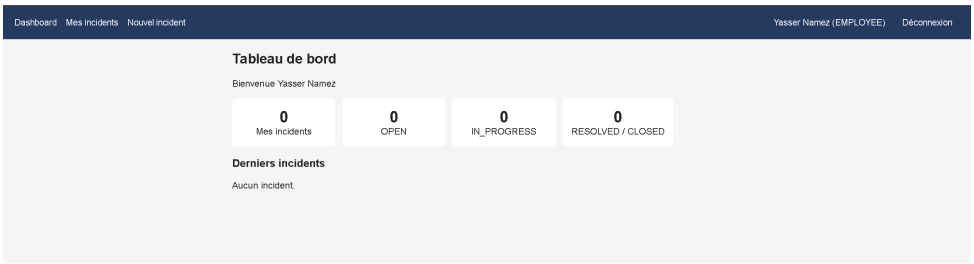


FIGURE 4 – Dashboard de l'employé

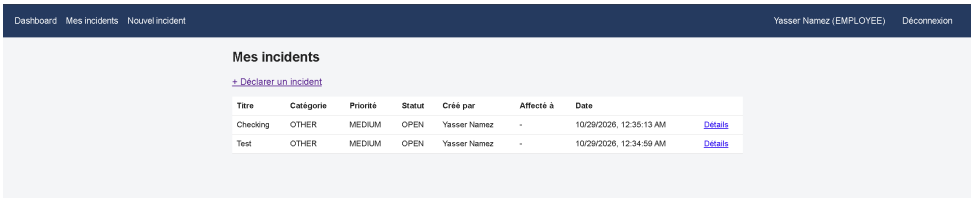


FIGURE 5 – Déclaration d'un incident

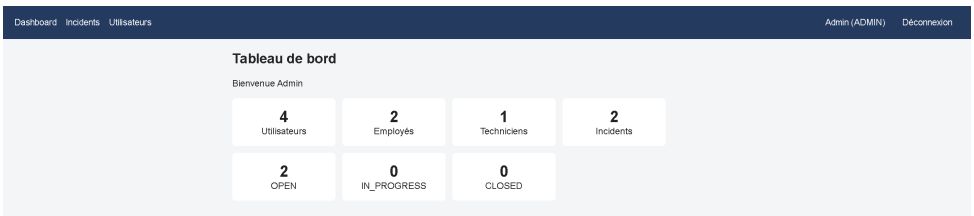


FIGURE 6 – Dashboard de l'administrateur

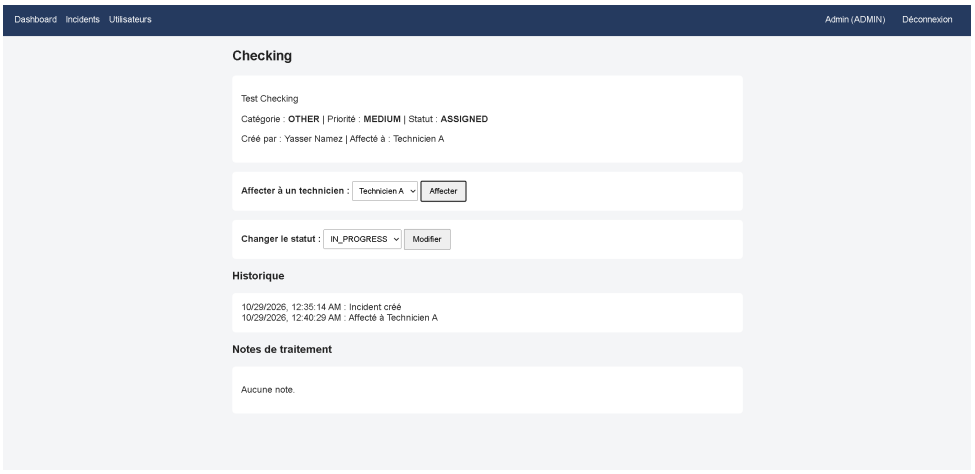


FIGURE 7 – Affectation d'un incident à un technicien

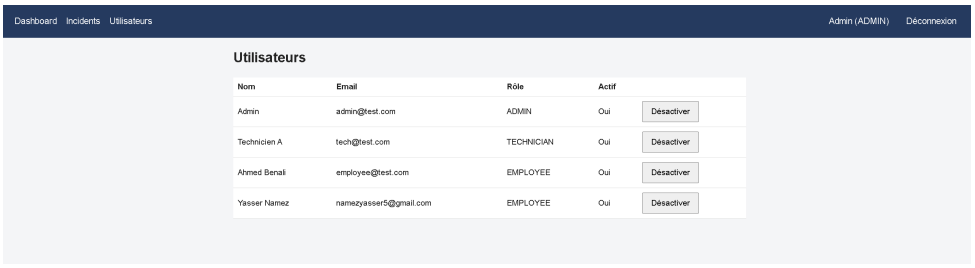


FIGURE 8 – Page de gestion des utilisateurs (ADMIN)

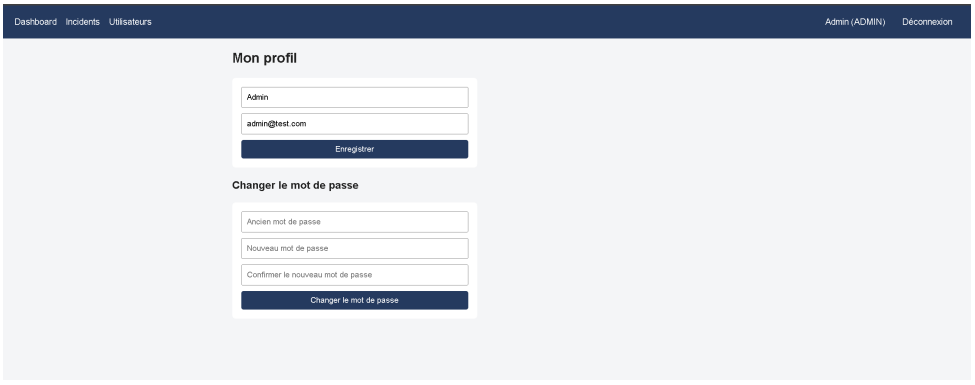


FIGURE 9 – Page de profil

12.2 Tests de sécurité avec Postman

N°	Test	Résultat attendu
1	API protégée sans JWT	401
2	JWT invalide	401
3	JWT expiré	401
4	EMPLOYEE sur une route ADMIN	403
5	EMPLOYEE sur l'incident d'un autre	403
6	TECHNICIAN sur un incident non affecté	403
7	Utilisateur désactivé qui se connecte	403
8	Inscription avec un email existant	409
9	Création d'un incident avec données invalides	400

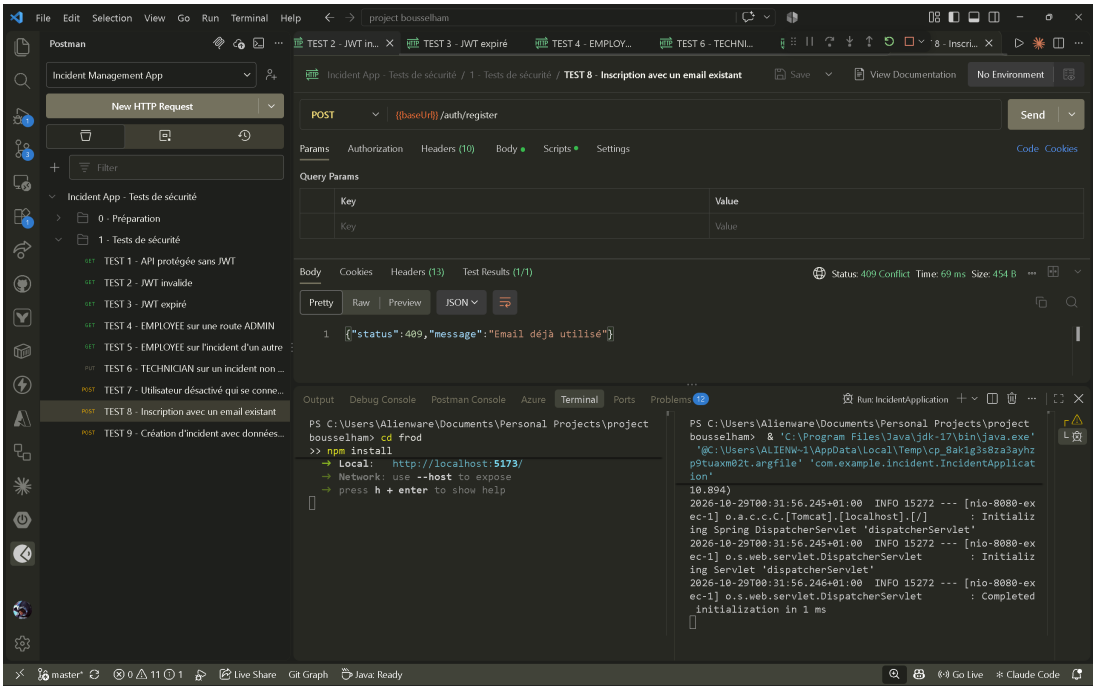


FIGURE 10 – Résultat des tests de sécurité (Collection Runner)